



Introduction to **JavaScript**

Presented by
Nguyễn Đức Thái
Ho Chi Minh City University of Technology

Outline

- Introduction
- History of JavaScript
- Hello World Web Page
- Buttons
- Functions
- Variables
- Identifiers

Introduction

- JavaScript is the *world's most popular* programming language.
- JavaScript is the programming language *of the Web*.
- JavaScript is *easy to learn*.
- JavaScript is *one* of the *3 languages* all web developers **must learn**:
 - HTML to *define the content* of web pages
 - CSS to specify the *layout* of web pages
 - JavaScript to program the *behavior* of web pages

What Can JavaScript Do?

- JavaScript can:
 - Change HTML Content
 - Change HTML Attribute Values
 - Change HTML Styles (CSS)
 - Hide HTML Elements
 - Show HTML Elements

Change HTML Content

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>What Can JavaScript Do?</h2>
```

```
<p id="demo">JavaScript can change HTML content.</p>
```

```
<button type="button"
```

```
onclick='document.getElementById("demo").innerHTML = "Hello  
JavaScript!'">Click Me!</button>
```

```
</body>
```

```
</html>
```

What Can JavaScript Do?

JavaScript can change HTML content.

Click Me!

What Can JavaScript Do?

Hello JavaScript!

Click Me!

Change HTML Attribute Values

```
<h2>What Can JavaScript Do?</h2>
```

```
<p>JavaScript can change HTML attribute values.</p>
```

```
<p>In this case JavaScript changes the value of the src (source) attribute of an image.</p>
```

```
<button onclick="document.getElementById('myImage').src='pic_bulbon.gif'">
```

Turn on the light

```
</button>
```

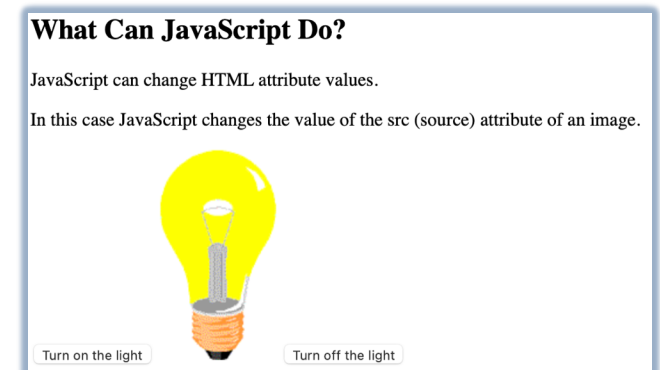
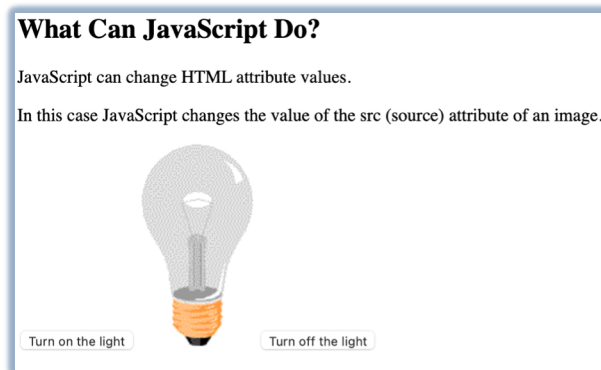
```

```

```
<button onclick="document.getElementById('myImage').src='pic_bulboff.gif'">
```

Turn off the light

```
</button>
```



Change HTML Styles (CSS)

Changing the style of an HTML element, is a variant of changing an HTML attribute:

What Can JavaScript Do?

JavaScript can change the style of an HTML element.

Click Me!

```
<!DOCTYPE html>
<html>
<body>
```

```
<h2>What Can JavaScript Do?</h2>
```

```
<p id="demo">JavaScript can change the style of an HTML element.</p>
```

```
<button type="button"
onclick="document.getElementById('demo').style.fontSize='35px'">Click Me!
</button>
```

```
</body>
</html>
```

What Can JavaScript Do?

JavaScript can change the style of an HTML element.

Click Me!

JavaScript Can Hide HTML Elements

Hiding HTML elements can be done by changing the display style:

```
<!DOCTYPE html>
<html>
<body>

  <h2>What Can JavaScript Do?</h2>

  <p id="demo">JavaScript can hide HTML elements.</p>

  <button type="button"
onclick="document.getElementById('demo').style.display='none'">Click Me!
</button>

</body>
</html>
```

What Can JavaScript Do?

JavaScript can hide HTML elements.

Click Me!

What Can JavaScript Do?

Click Me!

JavaScript Can Show HTML Elements

Showing hidden HTML elements can also be done by changing the display style:

```
<!DOCTYPE html>
<html>
<body>
```

```
<h2>What Can JavaScript Do?</h2>
```

```
<p>JavaScript can show hidden HTML elements.</p>
```

```
<p id="demo" style="display:none">Hello JavaScript!</p>
```

```
<button type="button"
onclick="document.getElementById('demo').style.display='block'">Click Me!
</button>
```

```
</body>
</html>
```

What Can JavaScript Do?

JavaScript can show hidden HTML elements.

Click Me!

What Can JavaScript Do?

JavaScript can show hidden HTML elements.

Hello JavaScript!

Click Me!

JavaScript Where To

- The <script> Tag
- JavaScript Functions and Events
- JavaScript in <head> or <body>
- JavaScript in <head>
- JavaScript in <body>
- External JavaScript
- External JavaScript Advantages

The <script> Tag

In HTML, JavaScript code is inserted between <script> and </script> tags.

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>JavaScript in Body</h2>
```

```
<p id="demo"></p>
```

```
<script>
```

```
document.getElementById("demo").innerHTML = "My First JavaScript";
```

```
</script>
```

```
</body>
```

```
</html>
```

JavaScript in Body

My First JavaScript

JavaScript in <head> or <body>

- You can place *any number of scripts* in an HTML document.
- Scripts can be placed **in the <body>**, or **in the <head>** section of an HTML page, or **in both**.

JavaScript in <head>

The function is invoked (called) when a button is clicked:

```
<!DOCTYPE html>
<html>
```

```
<head>
```

```
<script>
```

```
function myFunction() {
  document.getElementById("demo").innerHTML = "Paragraph changed.";
}
```

```
</script>
```

```
</head>
```

```
<body>
```

```
<h2>Demo JavaScript in Head</h2>
```

```
<p id="demo">A Paragraph</p>
```

```
<button type="button" onclick="myFunction()">Try it</button>
```

```
</body>
```

```
</html>
```



Demo JavaScript in Head

A Paragraph.

Try it

Demo JavaScript in Head

Paragraph changed.

Try it

JavaScript in <body>

The function is invoked (called) when a button is clicked:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>Demo JavaScript in Body</h2>
```

```
<p id="demo">A Paragraph.</p>
```

```
<button type="button" onclick="myFunction()">Try it</button>
```

```
<script>
```

```
function myFunction() {
```

```
    document.getElementById("demo").innerHTML = "Paragraph changed.";
```

```
}
```

```
</script>
```

```
</body>
```

```
</html>
```



Demo JavaScript in Body

A Paragraph.

Try it

Demo JavaScript in Body

Paragraph changed.

Try it

External JavaScript

External scripts are practical when the **same code is used in many different web pages.**

```
<!DOCTYPE html>
<html>
<body>
  <h2>Demo External JavaScript</h2>

  <p id="demo">A Paragraph.</p>

  <button type="button" onclick="myFunction()">Try it</button>

  <p>This example links to "myScript.js".</p>
  <p>(myFunction is stored in "myScript.js")</p>

  <script src="myScript.js"></script>

</body>
</html>
```

- You can place an external script reference in `<head>` or `<body>` as you like.
- The script will behave as if it was located exactly where the `<script>` tag is located.

External JavaScript Advantages

- Placing scripts in external files has *some advantages*:
 - It *separates* HTML and code
 - It makes HTML and JavaScript *easier to read* and maintain
 - Cached JavaScript files can *speed up* page **loads**

JavaScript Output

- JavaScript Display Possibilities
 - Using `innerHTML`
 - Using `document.write()`
 - Using `window.alert()`
 - Using `console.log()`
 - JavaScript **Print**

Using innerHTML

To access an HTML element, JavaScript can use **document.getElementById(id).**

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>My First Web Page</h2>
```

```
<p>My First Paragraph.</p>
```

```
<p id="demo"></p>
```

```
<script>
```

```
document.getElementById("demo").innerHTML = 5 + 6;
```

```
</script>
```

```
</body>
```

```
</html>
```

My First Web Page

My First Paragraph.

11

Using document.write()

For *testing purposes*, it is convenient to use `document.write()`:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
    <h2>My First Web Page</h2>
```

```
    <p>My first paragraph.</p>
```

```
    <p>Never call document.write after the document has finished loading.  
    It will overwrite the whole document.</p>
```

```
    <script>
```

```
        document.write(5 + 6);
```

```
    </script>
```

```
</body>
```

```
</html>
```

My First Web Page

My first paragraph.

Never call document.write after the document has finished loading. It will overwrite the whole document.

11

Using document.write()

Using `document.write()` after an HTML document is loaded, will **delete all existing HTML**:

```
<!DOCTYPE html>
<html>
<body>
```

```
    <h2>My First Web Page</h2>
    <p>My first paragraph.</p>
```

```
    <button type="button" onclick="document.write(5 + 6)">Try it</button>
```

```
</body>
</html>
```

My First Web Page

My first paragraph.

Try it

11

The document.write() method should only be used for testing.

Using `window.alert()`

You can use an *alert box* (popup) to display data:

```
<!DOCTYPE html>
<html>
<body>

  <h2>My First Web Page</h2>
  <p>My first paragraph.</p>

  <script>
    window.alert(5 + 6);
  </script>

</body>
</html>
```

My First Web Page

My first paragraph.

11

Close

You can skip the **window** keyword.

In JavaScript, the **window** object is the **global scope object**, that means that variables, properties, and methods *by default* **belong to the window object**. This also means that specifying the window keyword is **optional**

Using `console.log()`

For *debugging purposes*, you can call the `console.log()` *method* in the browser to display data.

```
<!DOCTYPE html>
<html>
<body>

    <h2>Activate Debugging</h2>

    <p>F12 on your keyboard will activate debugging.</p>
    <p>Then select "Console" in the debugger menu.</p>
    <p>Then click Run again.</p>

    <script>
        console.log(5 + 6);
    </script>

</body>
</html>
```

Activate Debugging

F12 on your keyboard will activate debugging.

Then select "Console" in the debugger menu.

Then click Run again.

JavaScript Print

JavaScript does not have any print object or print methods.

You cannot access output devices from JavaScript.

The only exception is that you can call the `window.print()` method in the browser to print the content of the current window.

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>The window.print() Method</h2>
```

```
<p>Click the button to print the current page.</p>
```

```
<button onclick="window.print()">Print this page</button>
```

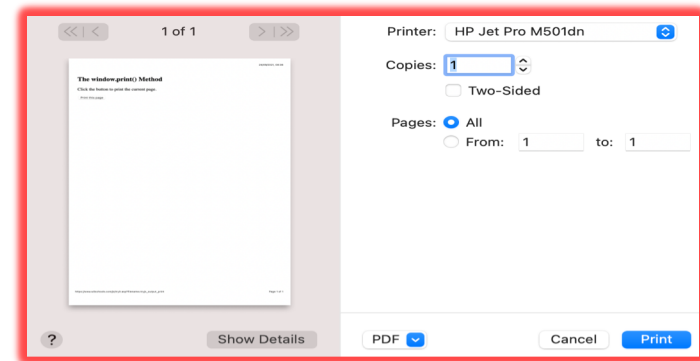
```
</body>
```

```
</html>
```

The window.print() Method

Click the button to print the current page.

Print this page



JavaScript Statements

- JavaScript *statements* are composed of:

- Values,
- Operators,
- Expressions,
- Keywords, and
- Comments.

```
<script>  
let x, y, z;    // Statement 1  
x = 5;         // Statement 2  
y = 6;         // Statement 3  
z = x + y;     // Statement 4
```

- A **computer program** is a list of "*instructions*" to be "*executed*" by a computer.
- In a programming language, these programming instructions are called *statements*.
- A JavaScript program is a list of *programming statements*.
- In HTML, **JavaScript programs** are executed by the **web browser**.

Semicolon

- **Semicolons** *separate* JavaScript statements.
- Add a semicolon **at the end** of each executable statement.

```
let a, b, c; // Declare 3 variables  
a = 5;      // Assign the value 5 to a  
b = 6;      // Assign the value 6 to b  
c = a + b;   // Assign the sum of a and b to c
```

- When separated by semicolons, *multiple statements* on **one line** are allowed.

```
a = 5; b = 6; c = a + b;
```

JavaScript White Space

- JavaScript **ignores** multiple spaces. You can add white space to your script to make it more readable.

```
let person = "Hege";  
let person="Hege";
```

- A *good practice* is to put spaces around operators (= + - * /):

```
let x = y + z;
```

JavaScript Line Length & Line Breaks

- For best readability, programmers often like to **avoid** code lines *longer than 80 characters*.
- If a JavaScript statement *does not fit* on one line, the best place *to break it is after an operator*:

```
document.getElementById("demo").innerHTML =  
"Hello Dolly!";
```

JavaScript Code Blocks

- JavaScript *statements* can be *grouped together* in code **blocks**, *inside curly brackets {...}*.
- The purpose of code blocks is to define statements to be *executed together*.

```
function myFunction() {  
    document.getElementById("demo1").innerHTML = "Hello Dolly!";  
    document.getElementById("demo2").innerHTML = "How are you?";  
}
```


JavaScript Keywords

- JavaScript *statements* often *start with* a **keyword** to identify the JavaScript action to be performed.

Keyword	Description
var	Declares a variable
let	Declares a block variable
const	Declares a block constant
if	Marks a block of statements to be executed on a condition
switch	Marks a block of statements to be executed in different cases
for	Marks a block of statements to be executed in a loop
function	Declares a function
return	Exits a function
try	Implements error handling to a block of statements

JavaScript Syntax

- JavaScript syntax is the *set of rules*, how JavaScript programs are constructed:

```
// How to create variables:  
var x;  
let y;  
  
// How to use variables:  
x = 5;  
y = 6;  
let z = x + y;
```

JavaScript Values

- The JavaScript *syntax* defines two types of values:
 - Fixed values
 - Variable values
- Fixed values are called **Literals**.
- Variable values are called **Variables**.

JavaScript Comments

- JavaScript *comments* can be used *to explain JavaScript code*, and *to make it more readable*.
- JavaScript comments can also be used to *prevent execution*, when testing alternative code.

```
let x = 5;      // Declare x, give it the value of 5  
let y = x + 2; // Declare y, give it the value of x + 2
```

- Multi-line comments start with `/*` and end with `*/`.

```
/*  
The code below will change  
the heading with id = "myH"  
and the paragraph with id = "myP"  
in my web page:  
*/
```

JavaScript Variables

- **3 ways** to declare a JavaScript *variable*:
 - Using **var**
 - Using **let**
 - Using **const**
- Variables defined with **let** cannot be **re-declared**.
- Variables defined with **let** *must be declared before use*.
- Variables defined with **let** have **Block Scope**.

```
var x = 5;  
var y = 6;  
var z = x + y;
```

JavaScript Const

- Variables defined with **const** *cannot be re-declared*.
- Variables defined with **const** *cannot be re-assigned*.

```
const PI = 3.141592653589793;  
PI = 3.14;           // This will give an error  
PI = PI + 10;        // This will also give an error
```
- Variables defined with **const** have *block scope*.
- JavaScript **const** variables *must be assigned* a **value** when they are declared

When to Use JavaScript Const?

- *As a general rule, always declare a variable with **const** unless you know that the value will change.*
- Use **const** when you declare:
 - A new Array
 - A new Object
 - A new Function
 - A new RegExp

Constant Arrays

- Can *change* the elements of a constant array:

```
// You can create a constant array:  
const cars = ["Saab", "Volvo", "BMW"];  
  
// You can change an element:  
cars[0] = "Toyota";  
  
// You can add an element:  
cars.push("Audi");
```

- *Cannot re-assign* the array

```
const cars = ["Saab", "Volvo", "BMW"];  
  
cars = ["Toyota", "Volvo", "Audi"];    // ERROR
```


JavaScript Operators

- Arithmetic operators:

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
**	Exponentiation (<u>ES2016</u>)
/	Division
%	Modulus (Division Remainder)
++	Increment
--	Decrement

JavaScript Assignment Operators

- Arithmetic operators assign values

Operator	Example	Same As
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y
**=	x **= y	x = x ** y

JavaScript String Operators

- The **+** operator can also be used to add (concatenate) strings.

```
let text1 = "John";  
let text2 = "Doe";  
let text3 = text1 + " " + text2;
```

- The **+=** assignment operator can also be used to add (concatenate) strings:

```
let text1 = "What a very ";  
text1 += "nice day";
```

- Adding Strings and Numbers

```
let x = 5 + 5;  
let y = "5" + 5;  
let z = "Hello" + 5;
```

```
10  
55  
Hello5
```

JavaScript Comparison Operators

Operator	Description
==	equal to
===	equal value and equal type
!=	not equal
!==	not equal value or not equal type
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to
?	ternary operator

JavaScript Logical Operators

■ JavaScript Logical Operators

Operator	Description
&&	logical and
	logical or
!	logical not

■ JavaScript Type Operators

Operator	Description
typeof	Returns the type of a variable
instanceof	Returns true if an object is an instance of an object type

JavaScript Data Types

- JavaScript variables can hold different data types: *numbers*, *strings*, *objects* and *more*.

```
let length = 16;           // Number
let lastName = "Johnson";  // String
let x = {firstName:"John", lastName:"Doe"}; // Object
```

- Add Numbers and Strings

```
let x = 16 + "Volvo";
```

```
let x = 16 + 4 + "Volvo";
```

20Volvo

```
let x = "Volvo" + 16;
```

```
let x = "Volvo" + 16 + 4;
```

Volvo164

JavaScript Strings

- A string is *a series of characters* like "John Doe".
- Strings are written with quotes:

single or *double* quotes

```
let carName1 = "Volvo XC60";    // Using double quotes
let carName2 = 'Volvo XC60';    // Using single quotes
```

- Can use quotes *inside* a string.

```
let answer1 = "It's alright";    // Single quote inside double quotes
let answer2 = "He is called 'Johnny'"; // Single quotes inside double quotes
let answer3 = 'He is called "Johnny"'; // Double quotes inside single quotes
```

JavaScript Numbers

- JavaScript has only **one type** of *numbers*.
- Numbers can be written **with**, or **without** decimals:

```
let x1 = 34.00;    // Written with decimals  
let x2 = 34;       // Written without decimals
```

- *Extra large* or *extra small* numbers can be written with scientific (exponential) notation:

```
let y = 123e5;     // 12300000  
let z = 123e-5;    // 0.00123
```


JavaScript Booleans

- Booleans: **true** or **false**.

```
let x = 5;  
let y = 5;  
let z = 6;  
(x == y)      // Returns true  
(x == z)      // Returns false
```

- Booleans: often used *in* conditional testing.

JavaScript Arrays

- JavaScript arrays: **written** with *square brackets*.
- Array items: **separated** by *commas*.

```
const cars = ["Saab", "Volvo", "BMW"];
```

- Array indexes are **zero-based**:
 - first item is [0],
 - second is [1],
 - ... and so on.

JavaScript Objects

- JavaScript objects: **written** with *curly braces* {}.
- Object properties: **written** as
 - **name**: **value** *pairs*,
 - **separated** by *commas*.

```
const person = {firstName:"John", lastName:"Doe", age:50, eyeColor:"blue"};
```

JavaScript Functions

- A JavaScript **function** is a *block of code* designed to perform a particular task.
- A JavaScript **function** is *executed* when "something" invokes it (calls it).

```
function myFunction(p1, p2) {  
    return p1 * p2;    // The function returns the product of p1 and p2  
}
```

JavaScript Functions Syntax

- A JavaScript function: *defined* with the **function** *keyword*, followed by a *name*, followed by *parameters* ().
- Function names: *letters*, *digits*, *underscores*, and *dollar signs* (same rules as variables).

```
function name(parameter1, parameter2, parameter3) {  
    // code to be executed  
}
```

- Parameter names separated by commas:
(*parameter1*, *parameter2*, ...)

Functions Invocation

- The **code** *inside* the function will execute when "something" invokes (calls) the function:
 - When an **event** *occurs* (when a user clicks a button)
 - When it is *invoked* (called) from JavaScript code
 - *Automatically* (self invoked)

Functions Return

- When JavaScript *reaches* a **return** *statement*, the function will **stop executing**.
- If the *function was invoked* from a statement, JavaScript will "return" to execute the code after the invoking statement.
- Functions often compute a return value. The return value is "returned" *back to the "caller"*:

Why Functions?

- Can **reuse code**: define the code once, and use it many times.
- Can use the same code *many times* with different arguments, to *produce* different results.

Local Variables

- **Variables** declared within a JavaScript function, become **LOCAL** to the function.
- Local variables can only be accessed from within the function.

```
// code here can NOT use carName

function myFunction() {
  let carName = "Volvo";
  // code here CAN use carName
}

// code here can NOT use carName
```

References

- W3 Schools – [w3schools.com](https://www.w3schools.com)